

Student 13 **Kondapalli Pujitha** 192524188RC

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```

a = list(map(int, input().split()))
n = len(a)
for i in range(n):
    min_index = i
    for j in range(i + 1, n):
        if a[j] < a[min_index]:
            min_index = j
    a[i], a[min_index] = a[min_index], a[i]
print(a)

```

INPUT

23 78 45 8 32 56

OUTPUT

[8, 23, 32, 45, 56, 78]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```

a = list(map(int, input().split()))
for i in range(1, len(a)):
    key = a[i]
    j = i - 1
    while j ≥ 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
print(a)

```

INPUT

8 5 7 1 9 3

OUTPUT

[1, 3, 5, 7, 8, 9]

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
arr = list(map(int, input().split()))
print(merge_sort(arr))
```

INPUT

38 27 43 3 9 82 10

OUTPUT

[3, 9, 10, 27, 38, 43, 82]

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
import json
matrix = json.loads(input())
n = len(matrix)
diagonal = n == len(matrix[0]) and all(
    matrix[i][j] == 0 for i in range(n) for j in range(n) if i ≠ j)
print("Diagonal Matrix" if diagonal else "Not Diagonal Matrix")
```

INPUT

[[1,0,0],[0,5,0],[0,0,8]]

OUTPUT

Diagonal Matrix

Q5. Row and Column Sums

Score: 0.00 TC: 0/3 passed

CODE

```
matrix = eval(input())
rows = [sum(row) for row in matrix]
cols = [sum(matrix[i][j] for i in range(len(matrix))) for j in range(len(matrix[0]))]
print("Row:", rows)
print("Col:", cols)
```

INPUT

[[1,2],[3,4]]

OUTPUT

Security Error: Disallowed code pattern detected.

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
import sys
arr = list(map(int, sys.stdin.read().split()))
max_sum = arr[0]
current_sum = arr[0]
for num in arr[1:]:
    current_sum = max(num, current_sum + num)
    max_sum = max(max_sum, current_sum)
print(max_sum)
```

INPUT

1 2 3 4

OUTPUT

10

Q7. Common Elements Among Three Lists

Score: 0.00 TC: 0/3 passed

CODE

```
a = list(map(int, input().split()))
b = list(map(int, input().split()))
c = list(map(int, input().split()))
result = list(set(a) & set(b) & set(c))
print(result)
```

INPUT

1 2
3 4
5 6

OUTPUT

[]

Q8. Matrix Multiplication

Score: 0.00 TC: 0/3 passed

CODE

```
import json
A = json.loads(input())
B = json.loads(input())
rows = len(A)
cols = len(B[0])
n = len(B)
result = [[0] * cols for _ in range(rows)]
for i in range(rows):
    for j in range(cols):
        for k in range(n):
            result[i][j] += A[i][k] * B[k][j]
for row in result:
    print(row)
```

INPUT

[[2]]
[[3]]

OUTPUT

[6]

Q9. Sort Rotate Min Max

Score: 0.00 TC: 0/3 passed

CODE

```
lst = list(map(int, input().split()))
k = int(input())
sorted_list = sorted(lst)
k = k % len(sorted_list)
rotated = sorted_list[-k:] + sorted_list[:-k] if k else sorted_list
print("Sorted =", sorted_list)
print("Rotated =", rotated)
print("Min =", min(sorted_list))
print("Max =", max(sorted_list))
```

INPUT

```
3 1 2
1
```

OUTPUT

```
Sorted = [1, 2, 3]
Rotated = [3, 1, 2]
Min = 1
Max = 3
```

Q10. Sort Matrix Search

Score: 0.00 TC: 0/3 passed

CODE

```
import re
s = input()
nums = list(map(int, re.findall(r'?\d+', s)))
n = int(len(nums) ** 0.5)
matrix = [nums[i*n:(i+1)*n] for i in range(n)]
find = nums[-1]
a = sorted(sum(matrix, []))
matrix = [a[i*n:(i+1)*n] for i in range(n)]
pos = None
for i in range(n):
    for j in range(n):
        if matrix[i][j] == find:
            pos = (i, j)
print("Sorted =", matrix)
if pos:
    print("Position:", pos)
else:
    print("Not Found")
```

INPUT

```
Matrix=[[5,1],[4,2]], Find=2
```

OUTPUT

```
Sorted = [[1, 2], [4, 5]]
Position: (0, 1)
```